Experiment No – 1

Implement Simple Linear Regression

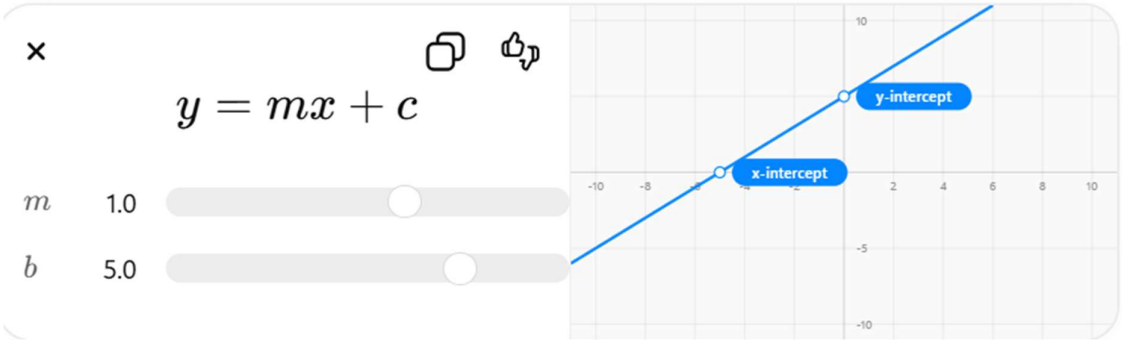
Aim

To implement Simple Linear Regression using Python and predict the relationship between two variables.

Theory

Simple Linear Regression is a supervised machine learning algorithm used to find the relationship between one independent variable (X) and one dependent variable (Y).
It predicts continuous numerical values by fitting a straight line between input and output data.

The mathematical equation of Simple Linear Regression is:



Where:

- **y** = predicted output
- **x** = input variable
- **m** = slope of line
- **c** = intercept

The algorithm tries to minimize the error between actual and predicted values using the best fit line.

Advantages

1. Simple and easy to understand
2. Fast training and prediction
3. Works well for linear data
4. Useful for trend prediction

Disadvantages

1. Works only for linear relationships
2. Sensitive to outliers
3. Cannot handle complex datasets properly
4. Accuracy decreases for non-linear data

Procedure

1. Import required libraries
2. Create input and output dataset
3. Split data into training and testing sets
4. Train the Linear Regression model
5. Predict output values
6. Display predicted result and graph

In [1]:

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Dataset
x = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 4, 5, 4, 5])

# Model training
model = LinearRegression()
model.fit(x, y)

# Prediction
y_pred = model.predict(x)

# Output
print("Predicted Values:", y_pred)

# Graph
plt.scatter(x, y, color='blue')
plt.plot(x, y_pred, color='red')
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Simple Linear Regression")
plt.show()
```

Result

Thus, Simple Linear Regression was implemented successfully using Python and the relationship between input and output variables was predicted using the best fit line.

Experiment No – 2

Implement Multiple Linear Regression

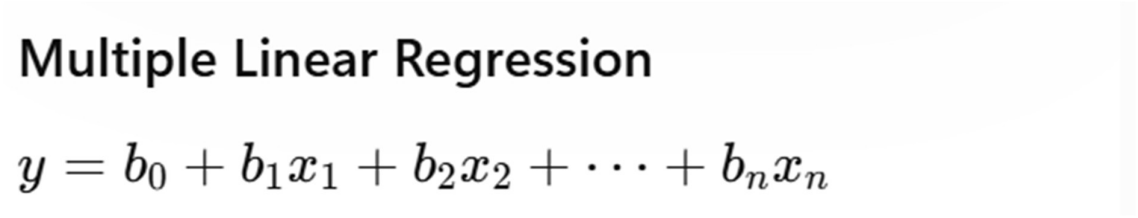
Aim

To implement Multiple Linear Regression using Python and predict output using multiple independent variables.

Theory

Multiple Linear Regression is a supervised machine learning algorithm used to predict a dependent variable using two or more independent variables.
It extends Simple Linear Regression by considering multiple input features together for prediction.

The mathematical equation of Multiple Linear Regression is:



Where:

- **y** = predicted output
- **b_0** = intercept
- **b_1, b_2 …** = coefficients
- **x_1, x_2 …** = input variables

This algorithm finds the best relationship between multiple inputs and output using a regression line in multi-dimensional space.

Advantages

1. Uses multiple features for prediction
2. Gives better accuracy than simple regression
3. Helps understand feature importance
4. Useful for real-world prediction problems

Disadvantages

1. Complex compared to simple regression
2. Sensitive to outliers
3. Requires large dataset for better accuracy
4. Multicollinearity can affect performance

Procedure

1. Import required libraries

2. Create dataset with multiple input variables
3. Train Multiple Linear Regression model
4. Predict output values
5. Display predicted results

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Dataset
X = np.array([[1,2],[2,3],[3,4],[4,5],[5,6]])
y = np.array([3,5,7,9,11])

# Model
model = LinearRegression()
model.fit(X, y)

# Graph
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X[:,0], X[:,1], y)

ax.set_xlabel("X1")
ax.set_ylabel("X2")
ax.set_zlabel("Y")

plt.title("Multiple Linear Regression")
plt.show()
```

Result

Thus, Multiple Linear Regression was implemented successfully using Python and output was predicted using multiple independent variables.

Experiment No – 3

Implement Logistic Regression

Aim

To implement Logistic Regression using Python for classification of data into different classes.

Theory

Logistic Regression is a supervised machine learning classification algorithm used to classify data into categories such as 0 or 1, Yes or No, True or False.
It predicts probability using the **sigmoid function** and converts it into class labels.

The sigmoid function is:

$$P(y) = \frac{1}{1+e^{-x}}$$

Where:

- **P(y)** = probability value
- **x** = input variable

If the probability is greater than 0.5, output becomes 1, otherwise 0.

Advantages

1. Easy to implement
2. Fast and efficient
3. Good for binary classification
4. Gives probability output

Disadvantages

1. Cannot solve complex non-linear problems
2. Sensitive to outliers
3. Requires clean data
4. Lower accuracy for complex datasets

Procedure

1. Import required libraries
2. Create dataset
3. Train Logistic Regression model
4. Predict output class
5. Plot graph for visualization

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression

# Dataset
X = np.array([[1], [2], [3], [4], [5], [6]])
y = np.array([0, 0, 0, 1, 1, 1])

# Model
model = LogisticRegression()
model.fit(X, y)

# Prediction
pred = model.predict([[3.5]])
print("Predicted Class:", pred)

# Graph
plt.scatter(X, y, color='blue')

x_test = np.linspace(1, 6, 100).reshape(-1,1)
y_prob = model.predict_proba(x_test)[:,1]

plt.plot(x_test, y_prob, color='red')

plt.xlabel("Input")
plt.ylabel("Probability")
plt.title("Logistic Regression")
plt.show()
```

Result

Thus, Logistic Regression was implemented successfully using Python and classification was visualized using a graph.

Experiment No – 4

Implement Decision Tree as Classification

Aim

To implement Decision Tree Classification using Python for classifying data into different categories.

Theory

Decision Tree is a supervised machine learning algorithm used for classification and prediction.
It works like a tree structure where:

- root node represents dataset
- branches represent conditions
- leaf nodes represent output classes

The model splits data based on important features to make decisions.

Decision Tree mainly uses conditions to classify data step by step until the final class is obtained.

Advantages

1. Simple and easy to understand
2. Works for both small and large datasets
3. Handles categorical and numerical data
4. Easy visualization using tree structure

Disadvantages

1. Can overfit the data
2. Accuracy decreases with noisy data
3. Complex trees become difficult to manage
4. Training may become slow for huge datasets

Procedure

1. Import required libraries
2. Create dataset for classification
3. Train Decision Tree Classifier
4. Predict output class
5. Visualize Decision Tree graph

```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Dataset
X = [[18], [22], [25], [35], [45], [50]]
y = ["Student", "Student", "Employee",
     "Employee", "Manager", "Manager"]

# Model
model = DecisionTreeClassifier()
model.fit(X, y)

# Prediction
pred = model.predict([[40]])
print("Predicted Role:", pred)

# Visualization
plt.figure(figsize=(8,5))
plot_tree(model,
          feature_names=["Age"],
          class_names=["Student", "Employee", "Manager"],
          filled=True)

plt.title("Decision Tree Classification")
plt.show()
```

Result

Thus, Decision Tree Classification was implemented successfully using Python and classification results were visualized using a decision tree graph.

Experiment No – 5

Implement Decision Tree as Regression

Aim

To implement Decision Tree Regression using Python for predicting continuous values.

Theory

Decision Tree Regression is a supervised machine learning algorithm used to predict continuous numerical values.
It works by dividing the dataset into smaller parts based on conditions and predicts output values using a tree-like structure.

Unlike Decision Tree Classification, regression predicts numerical outputs instead of categories.

The model creates decision rules based on input features and gives predicted values at leaf nodes.

Advantages

1. Easy to understand and implement
2. Handles non-linear data well
3. No need for data normalization
4. Works for both small and large datasets

Disadvantages

1. Can overfit the data
2. Sensitive to small data changes
3. Complex trees reduce performance
4. Lower accuracy for very large datasets

Procedure

1. Import required libraries
2. Create dataset
3. Train Decision Tree Regressor model
4. Predict output values
5. Plot regression graph

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor

# Dataset
X = np.array([[1], [2], [3], [4], [5], [6]])
y = np.array([2, 4, 5, 4, 5, 6])

# Model
model = DecisionTreeRegressor()
model.fit(X, y)

# Prediction
pred = model.predict([[4.5]])
print("Predicted Value:", pred)

# Graph
plt.scatter(X, y, color='blue')
plt.plot(X, model.predict(X), color='red')

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Decision Tree Regression")
plt.show()
```

Result

Thus, Decision Tree Regression was implemented successfully using Python and predicted continuous values using regression graph visualization.